

LAB 06 - INDEXING

Juliana Sofía Novoa Solano
Miguel Francisco Vargas Contreras

Administración de Bases de Datos

Profesor: ANDRÉS CALDERÓN ROMERO

Pontificia Universidad Javeriana
Bogotá D.C.
2026

Introducción

La recuperación eficiente de datos es un aspecto fundamental en la gestión de bases de datos, y la indexación desempeña un papel crucial en la optimización del rendimiento de las consultas. En este laboratorio, exploraremos el impacto de la indexación en el tiempo de ejecución de las consultas mediante el análisis de estas antes y después de la creación de índices.

Para lograrlo, trabajaremos con un conjunto de datos simulado que contiene cuentas de usuario, hilos de conversación y publicaciones en un foro hipotético. Generaremos datos aleatorios para poblar las tablas y ejecutaremos un conjunto de consultas predefinidas para medir su rendimiento. Utilizando el comando EXPLAIN ANALYZE en PostgreSQL, evaluaremos los tiempos de ejecución de las consultas, compararemos los resultados y observaremos cómo la indexación mejora la eficiencia.

Summaries

Functional dependencies

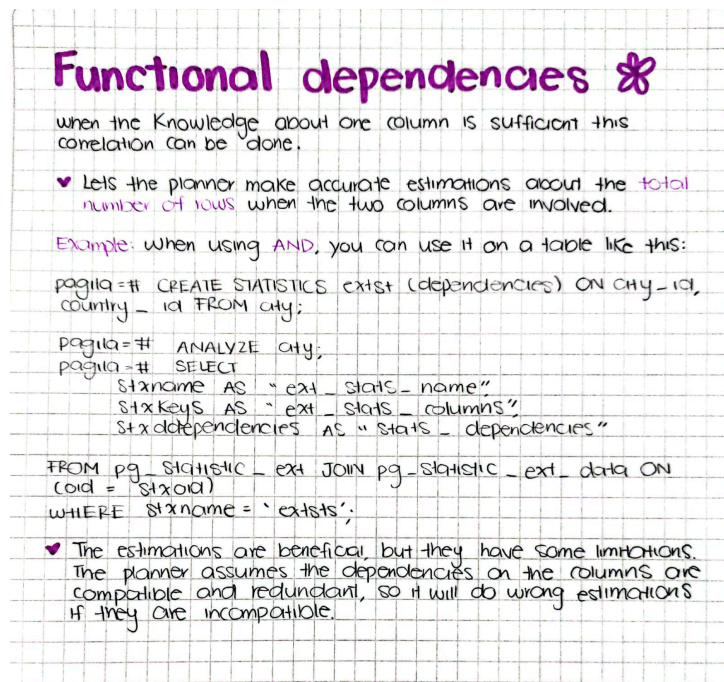


Figure 1: Functional dependencies summary.

Number of distinct values (NDV) counts

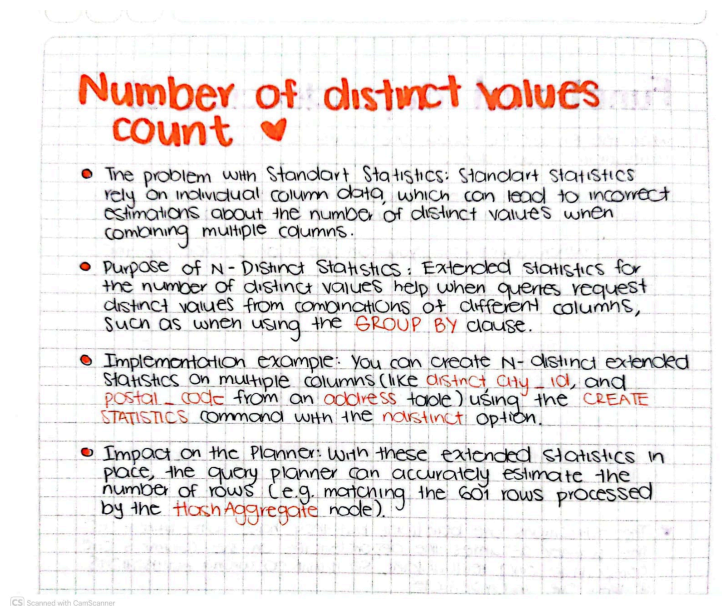


Figure 2: Number of distinct values counts summary.

Most common values (MCV) lists

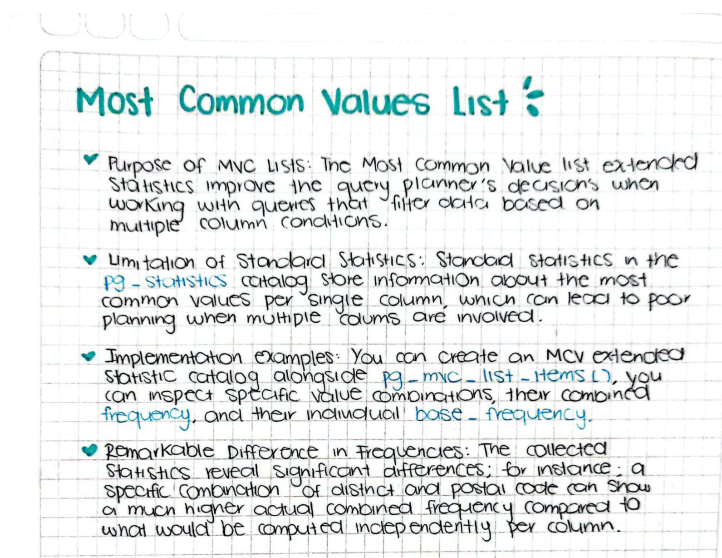


Figure 3: Most common values lists summary.

Data Generation and Performance Evaluation

En esta sección, estableceremos las bases prácticas para analizar el comportamiento del motor de la base de datos. El proceso se divide en dos etapas principales:

- **Generación de datos sintéticos:** Crearemos un esquema relacional básico y poblaremos las tablas con datos simulados utilizando funciones aleatorias y diccionarios de texto.
- **Evaluación de rendimiento:** Mediremos los tiempos de respuesta de consultas predefinidas antes y después de aplicar estrategias de indexación, utilizando la herramienta **EXPLAIN ANALYZE** para observar los cambios en los planes de ejecución.

Generating Random Data

```
acha@acha-macbook lab-6 % psql -h 100.117.32.119 -p 5555 -U acha -d lab6
Password for user acha:
psql (18.4, server 18.1 (Debian 18.1-1.pgdg13+2))
Type "help" for help.

lab6=#
```

Figure 4: Conexión a la db con psql-client.

```
lab6=# \i create.sql
psql:create.sql:1: NOTICE: drop cascades to 2 other objects
DETAIL: drop cascades to constraint thread_account_id_fkey on table thread
drop cascades to constraint post_account_id_fkey on table post
DROP TABLE
psql:create.sql:2: NOTICE: drop cascades to constraint post_thread_id_fkey on table post
DROP TABLE
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
lab6=#
```

Figure 5: Creación de tablas.

```
1 DROP TABLE IF EXISTS account CASCADE;
2 DROP TABLE IF EXISTS thread CASCADE;
3 DROP TABLE IF EXISTS post CASCADE;
4
5 CREATE TABLE account (
6     account_id SERIAL PRIMARY KEY,
7     name TEXT NOT NULL,
8     dob DATE
9 );
10
11 CREATE TABLE thread (
12     thread_id SERIAL PRIMARY KEY,
13     account_id INTEGER NOT NULL REFERENCES account(account_id),
14     title TEXT NOT NULL
15 );
16
17 CREATE TABLE post (
18     post_id SERIAL PRIMARY KEY,
19     thread_id INTEGER NOT NULL REFERENCES thread(thread_id),
20     account_id INTEGER NOT NULL REFERENCES account(account_id),
21     created TIMESTAMPT WITH TIME ZONE NOT NULL DEFAULT NOW(),
22     visible BOOLEAN NOT NULL DEFAULT TRUE,
23     comment TEXT NOT NULL
24 );
```

Figure 6: Creación de tablas (archivo).

```
acha@acha-macbook lab-6 % scp words.txt tally:/home/acha/college/acha
acha@acha-macbook:~$ scp words.txt tally:/home/acha/college/acha
words.txt
```

Figure 7: Mover el archivo words.txt al servidor.

```

services:
  postgres-tracker:
    image: postgres:18.1
    restart: always
    ports:
      - "5555:5432"
    environment:
      POSTGRES_USER: acha
      POSTGRES_PASSWORD: acha
      POSTGRES_DB: admin-db
    volumes:
      - ./db-data:/var/lib/postgresql
      - ./words.txt:/temp/words.txt

```

Figure 8: Edición del compose que crea el postgresql para poder tener el archivo.

```

lab6=# \i load_words.sql
DROP TABLE
CREATE TABLE
COPY 999

```

Figure 9: Carga del archivo al servidor.

```

load_words.sql
1  DROP TABLE IF EXISTS words CASCADE;
2
3  CREATE TABLE words (word TEXT);
4  COPY words(word) FROM '/temp/words.txt';

```

Figure 10: Crear la tabla “words” y llenarla con ese sql.

```

COPY 999
lab6=# \i random.sql
INSERT 0 100
INSERT 0 1000
INSERT 0 100000
lab6=#

```

Figure 11: Registros aleatorios en “account”, “thread” y “post”, creadas anteriormente.

```

random.sql
12
13 -- Creating 1000 random threads...
14 INSERT INTO thread (account_id, title)
15 SELECT
16     random() * 99 + 1,
17     (
18         SELECT
19             initcap(string_agg(word, ' '))
20         FROM
21             (TABLE words ORDER BY random() + n LIMIT 5) AS words (word)
22     )
23 FROM
24     generate_series(1, 1000) AS s(n);
25
26 -- Let's work with 100K random posts...
27 INSERT INTO post (thread_id, account_id, created, visible, comment)
28 SELECT
29     random() * 999 + 1,
30     random() * 99 + 1,
31     NOW() - ('1 days'::interval * random() * 1000),
32     CASE WHEN random() > 0.1 THEN TRUE ELSE FALSE END,
33     (
34         SELECT string_agg(word, ' ') FROM (TABLE words ORDER BY random() * n LIMIT 20) AS words(word)
35     )
36 FROM
37     generate_series(1, 100000) AS s(n);
38

```

Figure 12: Sql para creación de registros aleatorios.

```

1  -- Q1
2  EXPLAIN ANALYZE
3  SELECT post_id, thread_id, account_id, created, visible, comment
4  FROM post
5  WHERE account_id = 1;
6
7  -- Q2
8  EXPLAIN ANALYZE
9  SELECT COUNT(*) AS total_posts
10 FROM post
11 WHERE account_id = 1;
12
13 -- Q3
14 EXPLAIN ANALYZE
15 SELECT post_id, thread_id, account_id, created, comment
16 FROM post
17 WHERE thread_id = 1
18     AND visible = TRUE;
19
20 -- Q4
21 EXPLAIN ANALYZE
22 SELECT COUNT(*) AS total_posts
23 FROM post
24 WHERE account_id = 1
25     AND thread_id = 1;
26
27 -- Q5
28 EXPLAIN ANALYZE
29 SELECT post_id, thread_id, account_id, created, comment
30 FROM post
31 WHERE thread_id = 1
32     AND visible = TRUE
33     AND created ≥ date_trunc('month', CURRENT_TIMESTAMP)
34     AND created < date_trunc('month', CURRENT_TIMESTAMP) + INTERVAL '1 month'
35 ORDER BY created ASC;

```

Figure 13: Escritura de las 5 consultas.

Measuring Performance Without and With Indexes

See all my posts

```
lab6=# EXPLAIN ANALYZE
lab6=# SELECT post_id, thread_id, account_id, created, visible, comment
lab6=# FROM post
lab6=# WHERE account_id = 1;

QUERY PLAN

Seq Scan on post (cost=0.00..3395.00 rows=570 width=138) (actual time=0.279..16.232 rows=503.00 loops=1)
  Filter: (account_id = 1)
  Rows Removed by Filter: 99497
  Buffers: shared read=2145
Planning:
  Buffers: shared hit=74 read=21
Planning Time: 0.553 ms
Execution Time: 16.358 ms
(8 rows)
```

Figure 14: See all my posts query.

How many posts have I made?

```
lab6=# EXPLAIN ANALYZE
lab6=# SELECT COUNT(*) AS total_posts
lab6=# FROM post
lab6=# WHERE account_id = 1;

QUERY PLAN

Aggregate (cost=3396.43..3396.44 rows=1 width=0) (actual time=7.512..7.513 rows=1.00 loops=1)
  Buffers: shared hit=2145
  -> Seq Scan on post (cost=0.00..3395.00 rows=570 width=0) (actual time=0.028..7.465 rows=503.00 loops=1)
    Filter: (account_id = 1)
    Rows Removed by Filter: 99497
    Buffers: shared hit=2145
Planning:
  Buffers: shared hit=3 read=3
Planning Time: 0.097 ms
Execution Time: 7.545 ms
(10 rows)
```

Figure 15: How many posts have I made? query.

See all current posts for a Thread

```
QUERY PLAN

Seq Scan on post (cost=0.00..3395.00 rows=89 width=137) (actual time=0.029..6.867 rows=53.00 loops=1)
  Filter: (visible AND (thread_id = 1))
  Rows Removed by Filter: 99947
  Buffers: shared hit=2145
Planning Time: 0.087 ms
Execution Time: 6.886 ms
(6 rows)
```

Figure 16: See all current posts for a Thread query.

How many posts have I made to a Thread?

```
lab6=# EXPLAIN ANALYZE
lab6=# SELECT COUNT(*) AS total_posts
lab6=# FROM post
lab6=# WHERE account_id = 1
lab6=# AND thread_id = 1;

QUERY PLAN

Aggregate (cost=3645.00..3645.01 rows=1 width=0) (actual time=5.989..5.989 rows=1.00 loops=1)
  Buffers: shared hit=2145
  -> Seq Scan on post (cost=0.00..3645.00 rows=1 width=0) (actual time=5.906..5.906 rows=0.00 loops=1)
    Filter: ((account_id = 1) AND (thread_id = 1))
    Rows Removed by Filter: 100000
    Buffers: shared hit=2145
Planning:
  Buffers: shared hit=1 read=6
Planning Time: 0.190 ms
Execution Time: 5.924 ms
(10 rows)
```

Figure 17: How many posts have I made to a Thread? query.

See all current posts for a Thread for this month, in order

```
lab6=# EXPLAIN ANALYZE
lab6=# SELECT post_id, thread_id, account_id, created, comment
lab6=# FROM post
lab6=# WHERE thread_id = 1
lab6=# AND visible = TRUE
lab6=# AND created >= date_trunc('month', CURRENT_TIMESTAMP)
lab6=# AND created < date_trunc('month', CURRENT_TIMESTAMP) + INTERVAL '1 month'
lab6=# ORDER BY created ASC;

QUERY PLAN

Sort (cost=4080.02..4089.02 rows=1 width=137) (actual time=5.319..7.112 rows=0.00 loops=1)
  Sort Key: created
  Sort Method: quicksort Memory: 2560
  Buffers: shared hit=2145
  -> Gather (cost=1000.00..4089.02 rows=1 width=137) (actual time=0.296..7.889 rows=0.00 loops=1)
    Workers Planned: 1
    Workers Launched: 1
    Buffers: shared hit=2145
    -> Parallel Seq Scan on post (cost=0.00..3889.71 rows=1 width=137) (actual time=3.288..3.388 rows=0.00 loops=1)
      Filter: (visible AND (thread_id = 1) AND (created >= date_trunc('month', CURRENT_TIMESTAMP)) AND (created < (date_trunc('month', CURRENT_TIMESTAMP) + '1 mon'::interval)))
      Rows Removed by Filter: 5000
      Buffers: shared hit=2145
Planning:
  Buffers: shared hit=13 read=3
Planning Time: 0.251 ms
Execution Time: 7.138 ms
(10 rows)
```

Figure 18: See all current posts for a Thread for this month, in order query.

Performance of the queries with the best index implementation

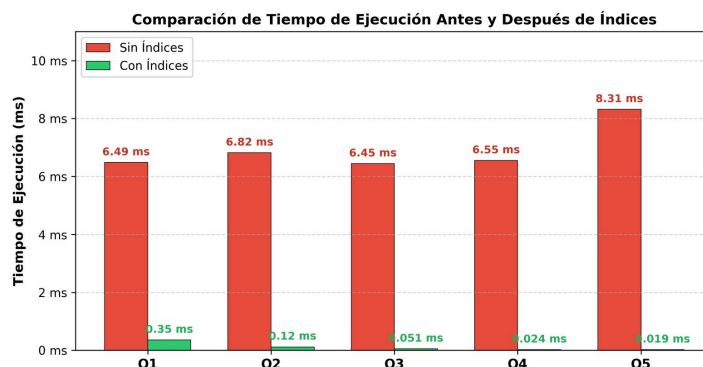


Figure 19: Comparación de tiempo de ejecución antes y después de Índices.

Análisis de Índices y Principio de Funcionamiento

El archivo `indexes.sql` propone una jerarquía de índices construidos sobre la tabla `post`. Cada uno de ellos ataca un patrón de acceso específico:

Índice Monocolumna: `CREATE INDEX ON post (account_id);`

- **Mecanismo B-Tree:** Organiza las claves de `account_id` de forma ordenada en páginas raíz, intermedias y hojas, junto con los identificadores físicos de tupla (`ctid`).
- **Mejora esperada:** Permite una búsqueda binaria $O(\log N)$ para ubicar las entradas correspondientes al usuario 1. Para **Q1**, PostgreSQL reemplaza el barrido completo de la tabla por un `Bitmap Index Scan`, reduciendo el tiempo de lectura.
- **Index Only Scan en Q2:** Dado que la consulta solo requiere contar filas y la columna `account_id` se encuentra almacenada directamente en el árbol, PostgreSQL (apoyándose en el **Visibility Map**) evita por completo acceder a la tabla base (**Heap Fetches: 0**), realizando una agregación directa en memoria.

Índice Monocolumna: `CREATE INDEX ON post (thread_id);`

- **Propósito:** Acelera búsquedas orientadas al identificador de hilo, aislando el 0.1% de posts asociados a un hilo específico dentro de los 100.000 registros.

Índices Compuestos: `(thread_id, visible)` y `(thread_id, account_id, visible)`

- **Orden de columnas y principio del prefijo izquierdo:** En un índice compuesto B-Tree, los datos se ordenan jerárquicamente por la primera columna, luego por la segunda y así sucesivamente.
- **Mejora en Q3 y Q4:** Permite satisfacer predicados de igualdad múltiples (`thread_id = 1 AND visible = TRUE` o `thread_id = 1 AND account_id = 1`) en una única pasada sobre el índice. Para **Q4**, el índice tricolumna cubre tanto el filtro como las columnas evaluadas, facilitando un `Index Only Scan` ultra rápido.

Índices Parciales: `(thread_id, account_id) WHERE visible = TRUE;`

- **Principio:** En lugar de indexar las 100.000 filas, solo se incluyen aquellas tuplas que cumplen el predicado `visible = TRUE`.

- **Ventajas:**

1. **Ahorro de almacenamiento y memoria:** Reduce el número de páginas hoja y el espacio ocupado en disco (1.7 MB vs 2.6 MB del índice completo), incrementando la probabilidad de permanencia en el **buffer pool**.
2. **Mayor densidad de claves:** Se eliminan del árbol las publicaciones ocultas o borradas lógicamente, acelerando las búsquedas de usuarios activos.

Índice Parcial con Ordenamiento: (thread_id, created) WHERE visible = TRUE;

- **Eliminación de la etapa de ordenamiento (Sort Elimination):** En la consulta **Q5**, sin índices el motor debe filtrar secuencialmente y luego ejecutar un algoritmo de ordenamiento en memoria (quicksort sobre disco/RAM). Con este índice, PostgreSQL busca las tuplas de thread_id = 1 y las recorre secuencialmente a través de las hojas del árbol B-Tree, las cuales ya están físicamente ordenadas por created ASC. Como consecuencia, el nodo Sort desaparece completamente del plan de ejecución.

Autonomous Work

MongoDB

Creación de índices



Figure 20: Índice en orden ascendente sobre el campo account_id.



Figure 21: Índice en orden ascendente sobre el campo thread_id.



```
await coll.createIndex(  
  { thread_id: 1, visible: 1 },  
  { name: "idx_thread_visible" },  
);
```

Figure 22: Índice que combina dos campos en orden ascendente: primero thread_id y luego visible.



```
await coll.createIndex(  
  { thread_id: 1, visible: 1 },  
  { name: "idx_thread_visible" },  
);
```

Figure 23: Índice que combina tres campos en orden ascendente: primero thread_id, luego account_id y visible.



```
await coll.createIndex(  
  { thread_id: 1, account_id: 1 },  
  {  
    partialFilterExpression: {  
      visible: true },  
    name: "idx_partial_thread_account",  
  },  
);
```

Figure 24: Crea un índice compuesto sobre thread_id y account_id, pero únicamente para los documentos donde visible: true.

```
await coll.createIndex(
  { thread_id: 1, created: 1 },
  {
    partialFilterExpression: {
      visible: true },
    name:
      "idx_partial_thread_created",
  },
);
```

Figure 25: Crea un índice compuesto agrupando por thread_id y ordenando por la fecha de creación (created), aplicando también el filtro parcial para incluir solo los registros con visible: true.

Comparación de Rendimiento MongoDB antes y después de índices

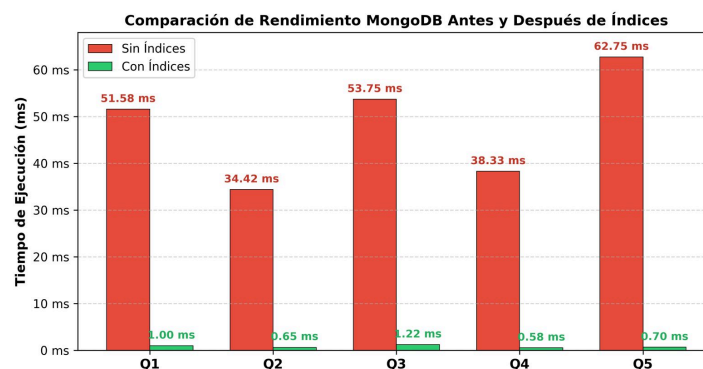


Figure 26: Rendimiento mongoDB.

- **sql/create.sql:** Sentencias DDL para crear las tablas relacionales requeridas: accounts, threads y posts.
- **sql/load_words.sql:** Instrucciones para cargar el diccionario de palabras a una tabla de apoyo en PostgreSQL.
- **sql/random.sql:** Procedimiento en PL/pgSQL que genera de forma sintética cuentas, hilos y 100.000 publicaciones con fechas y textos aleatorios.
- **sql/consultas.sql:** Definición de las 5 consultas de prueba (Q1 a Q5: posts de usuario, conteo de posts de usuario, posts de hilo, conteo de posts de hilo y posts mensuales ordenados).
- **sql/indexes.sql:** Creación de los índices B-Tree en PostgreSQL (idx_posts_account_id, idx_posts_thread_id y el índice compuesto idx_posts_thread_id_date).

- **src/index.ts**: Script de benchmark para PostgreSQL con Bun. Ejecuta Q1-Q5 sin índices y con índices, extrayendo tiempos promedio, desviación estándar, nodos de plan (Seq Scan, Bitmap Heap Scan, etc.) y accesos a buffers mediante EXPLAIN (ANALYZE, BUFFERS).
- **src/mongo_index.ts**: Script de benchmark para MongoDB (componente de trabajo autónomo NoSQL). Evalúa las 5 consultas equivalentes con .explain("executionStats") antes y después de crear índices, midiendo latencias y documentos examinados vs escaneados.
- **src/seed_mongo.ts**: Script para sembrar o migrar los 100.000 registros a MongoDB (ya sea directamente desde PostgreSQL o de forma autónoma).
- **src/plot.py**: Script en Python (Matplotlib) para generar las gráficas comparativas de rendimiento en formato PNG a partir de los JSON de métricas.
- **mongo/seed.js**: Script para poblar las colecciones accounts, threads y posts directamente en la consola mongosh.
- **mongo/consultas.js**: Sintaxis MQL de las 5 consultas con explicaciones de ejecución.
- **mongo/indexes.js**: Definición de índices en MongoDB (account_id, thread_id, compuesto con date).

Índices en postgresql vs mongo

Característica	PostgreSQL	MongoDB
Estructura Principal	Árboles B-Tree (además de GiST, GIN, BRIN y Hash).	Árboles B-Tree por defecto para índices secundarios y único (_id).
Creación de Índice	<pre>CREATE INDEX idx_name ON table_name(column_name);</pre>	<pre>await coll.createIndex({ field_name: 1 }, { name: "idx_name" });</pre>
Índices Parciales	<pre>CREATE INDEX idx_partial ON table(col) WHERE visible = true;</pre>	<pre>await coll.createIndex({ field: 1 }, { partialFilterExpression: { visible: true } });</pre>
Análisis de Rendimiento	EXPLAIN ANALYZE (muestra nodos como Seq Scan, Bitmap Index Scan y costos).	.explain("executionStats") (muestra documentos examinados vs. devueltos y latencia).

Conclusión

A lo largo de este laboratorio, se demostró de manera práctica el impacto crítico que tienen las estrategias de indexación en el rendimiento de los motores de bases de datos, abarcando tanto entornos relacionales (PostgreSQL) como no relacionales (MongoDB). A partir de la evaluación de consultas sobre un conjunto de datos sintético de 100,000 registros, se pueden extraer las siguientes conclusiones principales:

- **Mitigación del impacto del Seq Scan:** Sin índices, el motor de la base de datos se ve obligado a realizar un recorrido secuencial completo por toda la tabla o colección para resolver las consultas, lo que genera latencias elevadas y un consumo innecesario de recursos de E/S y memoria a medida que el volumen de datos crece.
- **Eficiencia de los índices B-Tree y compuestos:** La implementación de índices monocolumna y compuestos (como aquellos estructurados por `thread_id`, `account_id` y `visible`) permite reducir drásticamente los tiempos de ejecución de las consultas. En la mayoría de los casos evaluados (Q1 a Q5), el tiempo de respuesta disminuyó desde el rango de los milisegundos altos (6 ms - 50+ ms) hasta fracciones de milisegundo, transformando búsquedas costosas en accesos directos de tipo Index Scan.
- **Optimización mediante índices parciales y ordenamiento previo:** El uso de índices parciales (filtrados por condiciones como `visible = true`) no solo optimiza el espacio de almacenamiento y aumenta la densidad de claves en la memoria caché (buffer pool), sino que también permite la eliminación completa de fases de ordenamiento costosas (Sort Elimination), tal como se evidenció en la consulta Q5.
- **Paralelos conceptuales entre arquitecturas:** Aunque PostgreSQL y MongoDB manejan sintaxis y estructuras subyacentes distintas, comparten principios de optimización fundamentales. Mientras PostgreSQL aprovecha características avanzadas como Index Only Scans mediante mapas de visibilidad y planes analíticos detallados (`EXPLAIN ANALYZE`), MongoDB utiliza colecciones orientadas a documentos y colecciones indexadas con expresiones de filtros parciales (`partialFilterExpression`), logrando mejoras de rendimiento proporcionales y muy similares.

En resumen, una correcta planeación, diseño y aplicación de índices —alineados directamente con los patrones de acceso y las consultas más frecuentes de la aplicación— es una práctica indispensable para garantizar la escalabilidad, la eficiencia operativa y la reducción de la carga computacional en los sistemas gestores de bases de datos.